Australian Journal of Forensic Sciences

ISSN: 0045-0618 (Print) 1834-562X (Online) Journal homepage: www.tandfonline.com/journals/tajf20

An analysis of anti-forensic capabilities of B-tree file system (Btrfs)

Mohamad Ahtisham Wani, Wasim Ahmad Bhat & Ali Dehghantanha

To cite this article: Mohamad Ahtisham Wani, Wasim Ahmad Bhat & Ali Dehghantanha (2020) An analysis of anti-forensic capabilities of B-tree file system (Btrfs), Australian Journal of Forensic Sciences, 52:4, 371-386, DOI: 10.1080/00450618.2018.1533038

To link to this article: https://doi.org/10.1080/00450618.2018.1533038

Published online: 05 Nov 2018.

Submit your article to this journal

Article views: 410

View related articles

View Crossmark data

Citing articles: 6 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=tajf20

AUSTRALIAN JOURNAL OF FORENSIC SCIENCES
2020, VOL. 52, NO. 4, 371–386
https://doi.org/10.1080/00450618.2018.1533038

Taylor & Francis
Taylor & Francis Group

Check for updates

An analysis of anti-forensic capabilities of B-tree file system (Btrfs)

Mohamad Ahtisham Wani[a], Wasim Ahmad Bhat[a] and Ali Dehghantanha[b,c]

[a]Department of Computer Sciences, University of Kashmir, Srinagar, India; [b]Department of Computer Science, University of Sheffield, UK; [c]Cyber science Lab, School of Computer Science, University of Guelph, Guelph, Canada

ABSTRACT

Anti-forensic techniques aim to prevent, hinder or corrupt the forensic process of evidence acquisition, its analysis, and/or its admissibility. File systems are at the spotlight of almost every forensic investigation. The Linux B-tree file system (Btrfs) offers a paradigm shift in file system design by providing simple administration, end-to-end data integrity, and immense scalability without loss of performance. However, the potential of Btrfs for forensics examination and its resistance to anti-forensic activities was not investigated before. This paper covers this gap by analysing the forensics value of Btrfs and its robustness against anti-forensics activities. The experimental results suggest that Btrfs offers strong hurdles to many anti-forensic attacks. These include making it difficult to securely wipe files, disallowing hiding data in reserved locations of the file system data structures, and so on. Based on our findings, even a corrupt Btrfs volume could contain remnants of deletion of small files, hidden data in reserved locations and magic string forgery. Furthermore, forensic tools meant for Btrfs investigation must be augmented to support automated forensic analysis of possible hidden data in boot sector, file slack, volume slack and mount-point directories, MAC-DTS forgery, and sparse files.

ARTICLE HISTORY
Received 10 May 2018
Accepted 18 September 2018

KEYWORDS
Btrfs; anti-forensics; forensics; B-tree; file system

1. Introduction

Anti-forensic techniques aim to prevent, hinder or corrupt the forensic process of evidence acquisition, evidence analysis or presentation [1]. These techniques achieve their aim either by hiding or altering malicious remnants to avoid detection or make the data inadmissible in legal proceedings. Even though forensic techniques are evolving with time, anti-forensic techniques have also seen significant progress over time [2].

File systems are at the spotlight of (almost) every forensic investigation as they yield most of the digital evidences collected from a crime scene [3]. Consequently, file system anti-forensic techniques have witnessed significant growth over time so as to create maximum possible hindrance to the forensic investigation [4]. This makes it imperative for digital forensic investigators to understand anti-forensic capabilities of file systems for reliable and admissible crime scene investigation [5].

CONTACT Ali Dehghantanha dehqan@gmail.com Department of Computer Science, University of Sheffield, UK
© 2018 Australian Academy of Forensic Sciences

With the ever-increasing popularity of Linux as the main server-side operating system, tackling Linux scalability, reliability and security challenges has become important. To cope with these challenges, Linux offers a paradigm shift in file system design by providing simple administration, end-to-end data integrity, and immense scalability without loss of performance in the form of the B-tree file system (Btrfs) [6]. Btrfs is expected to work well on systems as small as a smartphone and as large as enterprise production servers [6]. This puts Btrfs at forefront of a forensics investigation in case of a breach or an intrusion. In addition, Btrfs is expected to be leveraged for malicious activities. However, there are very limited efforts for understanding the forensics value of Btrfs, especially in identification of anti-forensics activities and attempts [7].

This article presents an analysis of Btrfs in relation to investigation of anti-forensics activities. It employs well-established anti-forensic techniques to analyse how Btrfs data structures either facilitate or offer resistance to their successful execution, unveiling anti-forensic capabilities of Btrfs in the process. Based on the experimental results, it is found that those anti-forensic attacks that involve hiding data within reserved locations, securely removing small files, and attacking forensic tools by forging file system magic strings, result in B-tree file system volume corruption. Therefore, a corrupt Btrfs volume must be investigated specifically for such possible anti-forensic attacks during digital forensic investigation. Furthermore, strides must be made to advance forensic tools meant for Btrfs to support automated forensic investigation for other anti-forensic attacks, such as data hiding in the boot sector, file slack, volume slack and mount-point directories, MAC-DTS forgery, and use of sparse files.

The rest of this article is organized as follows. Section 2 discusses related work, Section 3 describes the methodology adopted, Section 4 discusses the results and Section 5 discusses the conclusion and makes some recommendations.

2. Related work

Cifuentes and Cano [8] proposed various anti-forensic techniques for Zettabyte File System (ZFS) based on its logical structure and on-disk layout, with the aim of unveiling its anti-forensic capabilities by analysing the resistance offered by its data structures to these anti-forensic attacks. The study also presents various recommendations to digital forensic investigators.

Srinivasan and Wu [9] employed a data-hiding anti-forensic technique on a FAT file system, proposing a file-hiding technique called *DupeFile*. The goal of the study was to highlight the vulnerability of FAT file systems to such anti-forensic attacks. This study was further extended by Srinivasan et al. [10] to investigate the *DupeFile* vulnerability of NTFS and HFS+ file systems. The study also proposed solutions to *DupeFile* vulnerability that can be used by file system forensic investigators in the form of *DupeFile Detector* and *DupeFile Extractor*.

Huebner et al. [11] showed how data structures specific to the NTFS file system facilitate data-hiding anti-forensic attacks in NTFS without being noticed by contemporary forensic tools and techniques. The study also shows techniques that ensure reliable recovery of the hidden data. Similarly, Cho [12] showed how slack spaces in NTFS's directory index can be exploited by data hiding anti-forensic attacks. In addition, Hsu et al. [13] highlighted NTFS vulnerability to sophisticated data-hiding anti-forensic

techniques that hide files by modifying their MFT record indexes and randomly rearranges the order of its data sectors. This makes it difficult even for sophisticated recovery tools to restore the contents of a hidden file.

Piper et al. [14] exploited reserved locations of various data structures of Ext2 and Ext3 file systems for a data-hiding anti-forensic technique, demonstrating its vulnerability to such anti-forensic attacks. The study proposed techniques to forensic investigators to detect anti-forensic attacks. Eckstein and Jahnke [15] showed how journal consistency checking mechanisms of journaling file systems such as the Ext3 file system are prone to anti-forensic attacks like data hiding. Gobel and Baier [16] found that data can be hidden in timestamp data structures of the Ext4 file system without being noticed by forensic tools and techniques.

Srinivasan et al. [17] demonstrated how encrypted data can be hidden in Linux file systems by generating slack space in cover files on demand.

Savoldi and Gubian [18] observed that SIM/USIM file systems are rich in undocumented files making the file system vulnerable to anti-forensic attacks such as data hiding.

Jang et al. [19] argued that timestamps in NTFS are vulnerable to timestamp forgery anti-forensic attack and proposed a forensic method to detect such manipulations. Lees [20] investigated the Update Sequence Number (USN) Journal file of NTFS to look for signs that indicate use of anti-forensic attacks on the file system. The study demonstrated the existence of patterns left by such anti-forensic activities within the USN Journal of NTFS that can reveal or detect use of possible anti-forensic attacks.

Since Btrfs is a recent addition to Linux file systems, no work to date exists that analyses Btrfs's logical structure and on-disk layout through forensic and anti-forensic perspectives.

3. Methodology

Previous works that aimed to analyse the anti-forensic capabilities of file systems [8,11] mainly exploited the design flaws of the file system. Since the aim of this paper is to perform the same analysis on Btrfs, we used a similar set of techniques, namely secured data-deletion, hiding data in reserved locations, like Superblock and boot-sector, hiding data in slack spaces like internal-node slack, file slack and volume slack, forging timestamps and modifying magic-numbers. We also used the sparse file feature and mounting on non-empty directory techniques on Btrfs to test if such techniques can hinder the working of forensic tools meant for Btrfs. A full description of these techniques can be found in Refs 21 and 22. To ensure that the best practices of digital forensics are employed, we adopted a forensic investigation framework that closely matches the one suggested in Ref. 23. Evidence was collected from the virtual disk drives of a 64bit VMWare Workstation version 12.1.1 build-3770994 running Fedora Core 23. Disk images were captured using dd utility. The MD5 hash value of each acquired file was calculated and subsequently verified. The captured images were subjected to rigorous forensic analysis using forensic tools and integrity checked using file system integrity checkers. They were also subjected to hexdump analysis to identify data remnants and digital evidences.

The experiments for anti-forensic analysis of Btrfs were performed on a 64- bit Fedora Core 23 Linux OS running on top of VMWare Workstation version 12.1.1 build-3770994 hosted on Windows 7 OS. Although Btrfs is not a default file system on Fedora Core 23, it was the only operating system at the time of research that supported Btrfs. To perform a

clean experiment, every time an anti-forensic technique was exercised on the file system, the volume was overwritten with zeros to ensure all data remnants are erased and then the Btrfs partition was created afresh. After successfully executing each anti-forensic procedure on the file system, the dd utility was used to capture the disk images. The experiments for forensic analysis of the captured disk images included their thorough forensic examination using forensic tools and integrity checking using file system checkers. Simple Linux utilities, such as wipe, dd, hexdump, touch, wipefs, fdisk, df, truncate, bmap and mount, were used to implement file system anti-forensic techniques. For forensic analysis, two forensic tools, Btrfs Forensic Master (BFM), and Access Data's FTK were used. BFM was the only open source forensic software at the time of research that could analyse Btrfs disk images and provide file browsing to some extent. In addition, it was the only available forensic tool that supported Btrfs. On the other hand, Access Data's FTK was the only commercial tool used in the experiments as it comes loaded with forensic utilities to suit different needs. In addition, FTK was used to arrive at a conclusion when BFM failed to generate any promising results. Furthermore, Btrfs's file system checker utility, btrfsck, was used in every experiment to verify the impact on file system integrity. Forensic analysis used the same setup as that of anti-forensic analysis except for the analysis performed using FTK, which was done using the Windows 7 host platform.

4. Results and discussion

This section discusses the results we collected from our experiments for every file system anti-forensic technique used.

4.1. Secure data-deletion

Conventionally, secure data-deletion is a straightforward procedure in most of the file systems [24–26]. However, this is not the case with Btrfs. We created a sample file *foo.txt* and employed wipe utility to securely delete it. When the disk image was analysed using BFM, no traces of *foo.txt* were found. However, hex and character-filtering utilities of FTK were able to locate many copies of the contents of the *foo.txt* file after searching for the file contents. This is because Btrfs uses the COW update model, which created copies of the file content during the wipe procedure and, possibly, previous file system checkpoints. To securely delete all the copies of the contents of the *foo.txt* file, we used the dd utility to directly overwrite the blocks containing the contents of the file. As expected, the file system mounted successfully, no traces of *foo.txt* were found during hexdump analysis and btrfsck verified the integrity of the file system. Since the size of *foo.txt* was larger than leaf size, the contents were stored in *regular-extents*. Interestingly, when the file is small enough to be stored in *inline-extent*, the secured data-deletion mechanism employed for *inline-extents* fails and results in file system corruption. This is because *inline-extents* are part of leaf nodes which are checksummed. Overwriting only *inline-extents* causes checksum mismatches, whereas overwriting whole leaf nodes result in loss of other information as well as causing checksum problems. This implies that secure data-deletion in Btrfs is a cumbersome task, and can only be achieved successfully for large files only, and that too only with skilled hands.

4.2. Reserved locations in Superblock

Btrfs's Superblock is located at an offset of 64 KB in to every block device and offers a total of 432 bytes reserved space, as shown in Figure 1.

Data from a sample file *foo.txt* was hidden in the reserved locations of Superblock using the dd command. When the disk image was analysed using BFM and FTK, no traces of hidden data were found. Also, btrfsck did not report any errors. Surprisingly, the hidden data are lost in subsequent re-mounts, or as and when the file system state changes. Since Superblock contains critical information about the overall file system and its state, whenever the file system is remounted or its state changes, the Superblock is reinitialized after every update to avoid checksum and generation mistakes. Consequently, it also removes any data in its reserved locations. This is in contradiction with the behaviour of other file systems, where even when Superblock is updated, reserved locations are not touched. This implies that Btrfs design is highly reluctant to allow data hiding in reserved locations of its Superblock. Furthermore, Superblock mirrors are kept at 64MB, 256GB, and 1PB in every block device, which are used when the first Superblock is infringed. These Superblocks can also be used for hiding data in the same manner as their primary counterpart. However, they report similar behaviour as Btrfs updates all Superblock copies when the primary Superblock gets updated.

4.3. Reserved locations in btrfs_root_items

Apart from Superblocks, reserved locations exist throughout Btrfs volume within btrfs_root_item data structures, as shown in Figure 2.

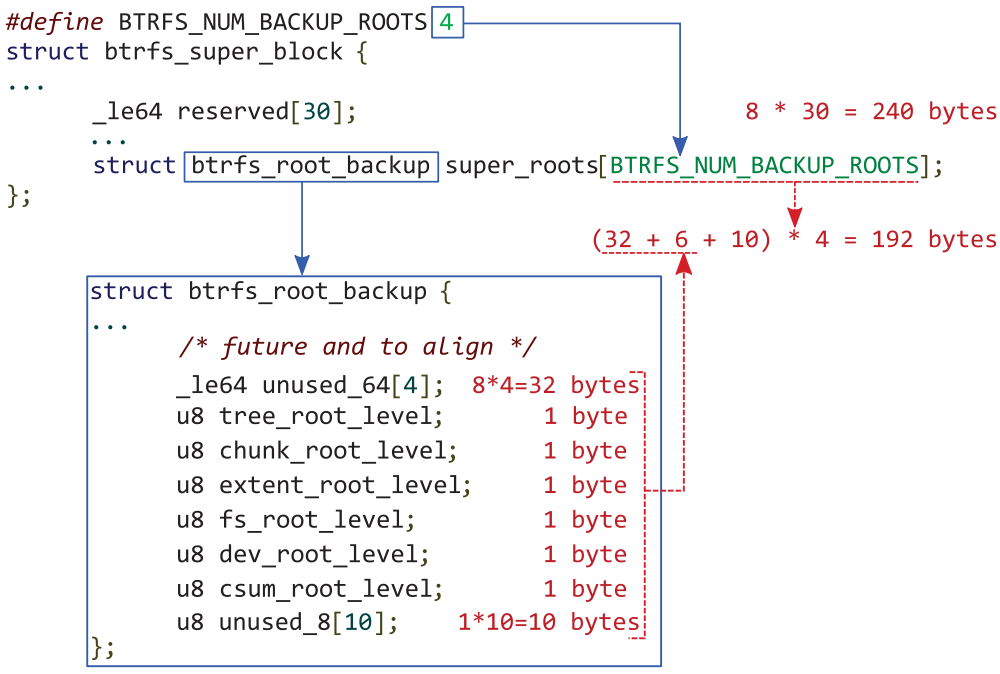

Figure 1. Reserved space available within the Superblock.

```
struct btrfs_root_item {
        struct btrfs_inode_item inode;          8 * 4 = 32 bytes
        ...
        __le64 reserved[8];                     8 * 8 = 64 bytes
};

        struct btrfs_inode_item {
        ...
                __le64 reserved[4];   8*4=32 bytes
                ...
        };
```

Figure 2. Reserved space available within btrfs_root_item.

When data were hidden within the reserved locations of these btrfs_root_items using the dd command, the file system was corrupted and became inaccessible during subsequent operation due to checksum mismatch errors. btrfsck was unable to recover the file system state and continuously reported checksum errors. All attempts to rescue the file system failed. However, BFM was able to traverse the file system but neither reported the presence of hidden data nor was able to detect any file system anomalies. On the other hand, FTK was able to mount the file system image as raw and revealed hidden data in its *hex* utility. This implies that, although hiding data within these locations of Btrfs may not serve any purpose, file system corruption due to checksum errors can induce significant delay, particularly in time-critical investigations, while keeping data hidden within btrfs_root_item.

4.4. Leaf slack

Btrfs's B-trees use leaf nodes to store actual information on files or directories, such as filenames, inode-data, *extent-data*, and so on. A leaf node stores an *item* array at the beginning, and a reverse sorted *item-data* array at the end in such a way that the two arrays grow towards each other as the leaf fills up, as shown in Figure 3.

Although, Btrfs ensures efficient space management, not all the free space that exists between the two arrays gets occupied completely due to the granularity of *items* and its associated data. This unoccupied space between *items* and *item-data (leaf slack)* is a characteristic feature of Btrfs. This space is small but occurs throughout the file system. Data was hidden in the leaf slack of file system B-tree using the dd command after unmounting the file system. Although data were hidden within the leaf slack successfully, the file system was unable to mount again, reporting checksum mismatch errors and file

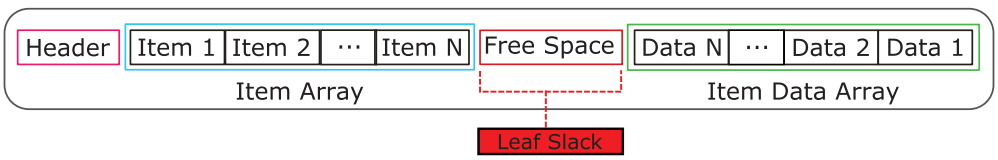

Figure 3. Structure of a leaf node showing leaf slack.

system corruption, and all attempts to repair the file system failed. In addition, forensic analysis by BFM and FTK also reported similar results. This implies that Btrfs's leaf slack may give an edge to criminals to induce delays in forensic investigation but it cannot be used to conceal data persistently and without causing file system anomalies.

4.5. Internal-node slack

Another file system slack space that is characteristic of Btrfs is the *internal-node slack*. This slack space exists within the internal nodes of Btrfs's B-trees which contain flat lists of *[key, block-pointer]* pairs, packed into the front of the node as shown in Figure 4.

Unlike leaf nodes, the remaining space in internal nodes is regarded as free space *(internal-node slack)*. In addition, due to the default block size of 4 KB, the amount of data that can be hidden is massive. However, data hiding within internal-node slack reported similar results as that of leaf slack, and resulted in file system corruption. This implies that the data hiding potential of internal-node slack cannot be exploited on Btrfs due to its sophisticated checksumming and scrubbing techniques and, hence, remains beyond reach.

4.6. Hiding data in the boot-sector

Btrfs reserves the first 64 kilobytes of each partition for the boot sector. Except for primary partitions, this space is not used by the file system and, as such, data concealed within these blocks does not cause any data overwrites or affect normal file system operations. In addition, its fixed location within the file system also makes it preferable to hide data both in terms of ease and available space. Data can be hidden in the boot sector of Btrfs using the dd utility. We used dd to hide some textual data in the boot sector of Btrfs partition as shown in Figure 5.

Unfortunately, forensic tools and file system checkers overlook the reserved locations of file systems during forensic analysis, which is what the result of our forensic analysis was in this case. Although FTK's *hex* and *character*-filtering utilities were able to display

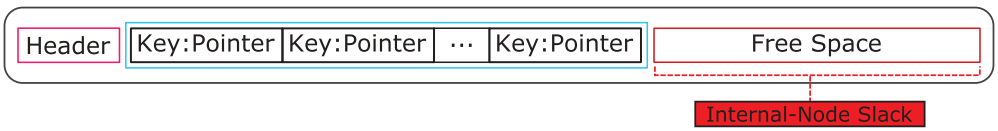

Figure 4. Structure of an internal node showing internal-node slack.

```
Boot
Sector
64 KB

00000000  54 68 69 73 20 69 73 20  74 68 65 20 68 69 64 64  |This is the hidd|
00000010  65 6e 20 64 61 74 61 20  73 74 6f 72 65 64 20 69  |en data stored i|
00000020  6e 20 74 68 65 20 42 6f  6f 74 20 20 53 65 63 74  |n the Boot  Sect|
*0000030  63 6b 20 6f 66 20 42 74  72 66 73 20 70 61 72 74  |or of Btrfs part|
00000040  69 74 69 6f 6e 2e 20 54  68 65 20 70 75 72 70 6f  |ition. The purpo|
*
0000fff0  73 74 65 6d 20 6f 70 65  72 61 74 69 6f 6e 73 2e  |stem operations.|
00010000
```

Figure 5. Hexdump of boot sector after hiding data in it.

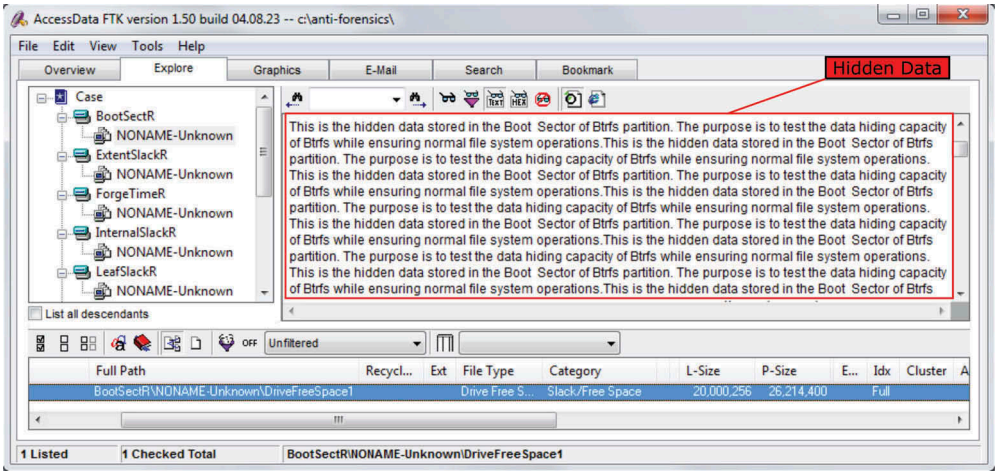

Figure 6. FTK view of data hidden in boot sector.

the contents of a boot sector as shown in Figure 6, such analysis techniques are limited to very small investigations as they do not scale well due to the intensive effort involved and time constraints of current digital forensic investigations. This implies that while it is trivial to hide data in a boot sector, Btrfs design does not create the hurdles that it created for Superblock, btrfs_root_item, leaf slack and internal-node slack.

4.7. File slack

Btrfs stores file *extents* either within *inline-extents* or *regular-extents*, which are composed of multiple contiguous blocks. However, file slack only exists in case of *regular-extents*. This is because, in case of *inline-extents*, files are stored as *[item, item-data]* pairs packed at the front and rear of leaf respectively, just like other metadata, making slack space impossible to exist. Although free space exists between *[item, item-data]* pairs at the middle of the leaf, such space cannot be used for persistent data hiding due to metadata checksums and redundancy. On the contrary, the amount of data that can be hidden within the file slack of a regular file *extent* is characterized by the block size and is always less than the actual size of the block. We created a sample file *foo.txt* with size 4.3 KB. As the file size is greater than one block, the file is stored in a *regular-extent*. The actual size and the number of blocks allocated to the file are shown in Figure 7.

Since the logical address of the *extent* is 4214784, and the size of the file is 4337 bytes, the starting address of file slack is 4219121. We used the dd utility to hide the data within this file slack. Figure 8 shows the hexdump of the file *foo.txt* after data are hidden within its slack space.

It is clear from Figures 7 and 8 that almost 0.7 KB of slack space is made available by the sample file *foo.txt*. After successfully hiding data within the available slack space, a disk image was captured and analysed using BFM and FTK. The BFM was unable to detect the presence of hidden data and did not report any file system anomalies. FTK, on the other hand, was able to reveal hidden data at the end of the file, as shown in Figure 9, when *character*-filtering was enabled in disk explorer. However, this is likely to

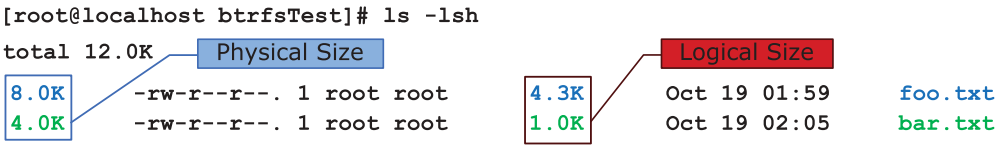

Figure 7. Physical and logical sizes of a 4.3 KB sample file *foo.txt*.

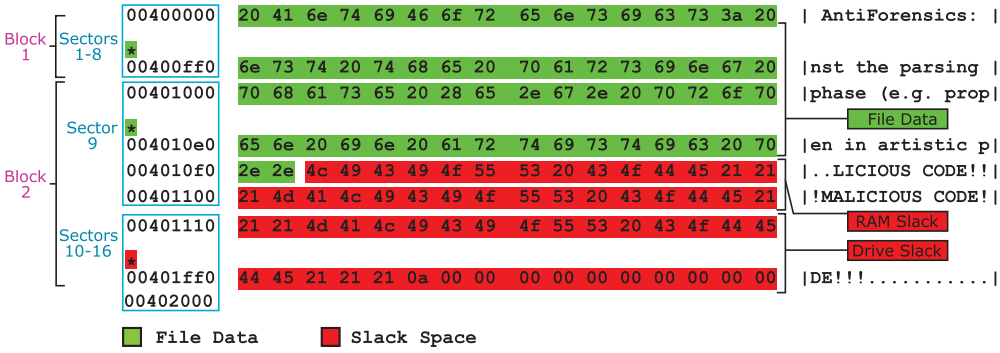

Figure 8. Hexdump of a sample file *foo.txt* and file slack.

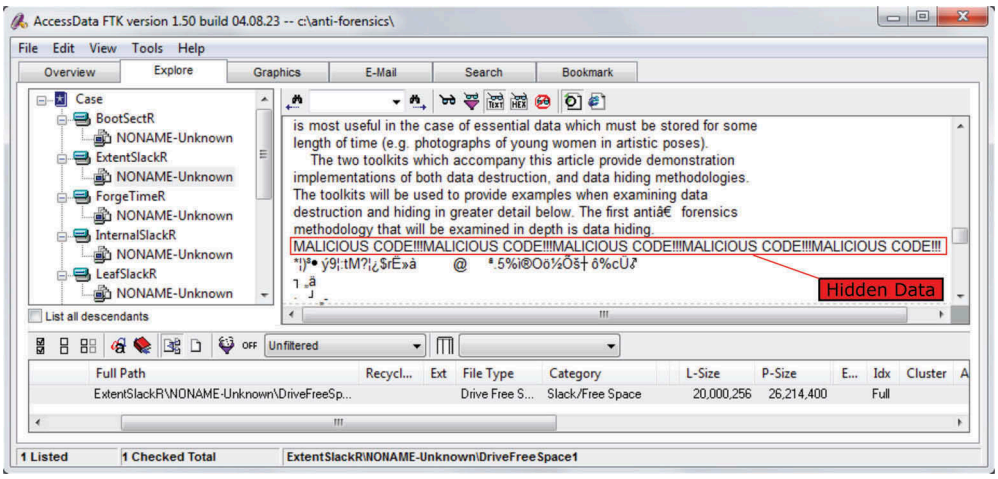

Figure 9. FTK view of data hidden in the file slack.

go undetected as forensic investigators use forensic tools for auto extraction and analysis whereas FTK's disk explorer requires manual analysis and extraction of forensic artefacts. Finally, the file system was checked using the btrfsck utility, which reported no file system inconsistencies and ended successfully without any errors. This is because information hidden in slack space is undetectable by file integrity checkers.

To complicate things, tools such as bmap, which are used to read slack space on most of the Linux file systems, fail miserably in case of Btrfs as they are unable to parse its data structures.

It can be easily argued that while file slack for files of size less than leaf size can't be used for hiding data, hidden data in file slack of larger files is neither denied by Btrfs nor identified by forensic tools. Since Btrfs is highly scalable and portable, this file slack offers huge space for data hiding across varied platforms for diverse workloads.

4.8. Volume slack

Volume slack is a characteristic feature of all file systems, and Btrfs is no exception. On a Btrfs volume, the number of sectors and 1K-blocks reported by fdisk and df have to be compared with block size in order to determine volume slack. We created a Btrfs partition with 39,063 sectors as shown in Figure 10.

We used the dd command to hide data in volume slack from sample file *bar.txt* as shown in Figure 11. With a block size of four sectors, the first 39,056 sectors are allocated to 4882 blocks and the last seven sectors remain unallocated to form the volume slack.

When the disk image was analysed using BFM, no traces of data hidden in the volume slack were found and btrfsck verified the integrity of file system. On the other hand, although FTK was able to reveal data hidden in the volume slack using its *character*-filtering utility as shown in Figure 12, it did not report its existence as being hidden in volume slack. This implies that volume slack is likely to go undetected on large storage systems managed by Btrfs. Furthermore, being purely unallocated as well as non-

```
[root@localhost ~]# fdisk -l /dev/sda4
Disk /dev/sda4: 19.1 MiB, 20000256 bytes, 39063 sectors
Units: sectors of 1 * 512 = 512 bytes

[root@localhost ~]# df -l /dev/sda4
Filesystem  1K-blocks  Used Available Use% Mounted on
/dev/sda4   19528      4128      15396  22% /run/media/root/test
```

Figure 10. fdisk and df showing number of sectors and 1 K-blocks.

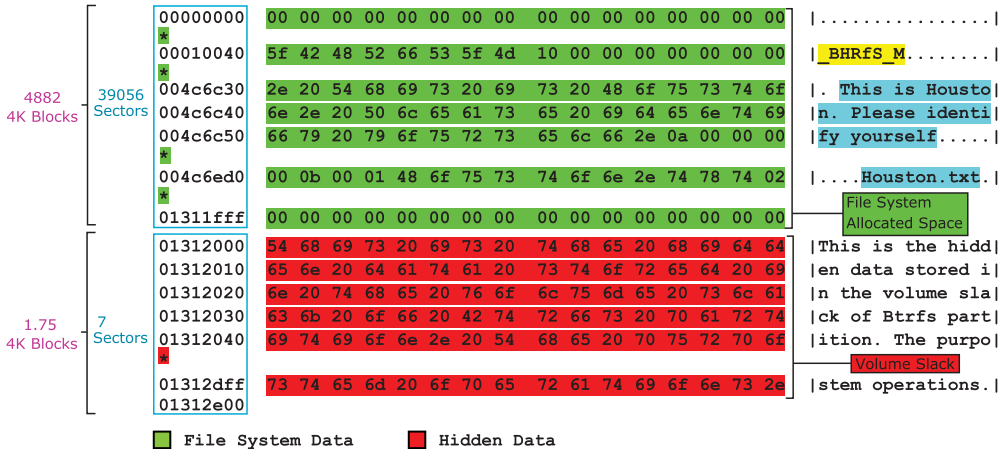

Figure 11. Hexdump of a sample file *bar.txt* and volume slack.

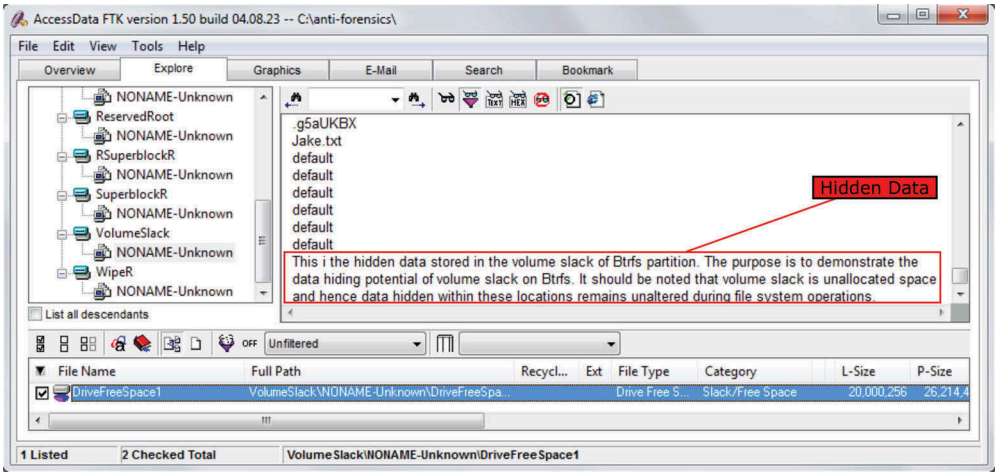

Figure 12. FTK view of data hidden in volume slack.

allocatable, volume slack provides the most secure location for hiding data. In summary, Btrfs design does not offer any hindrance to use volume slack for data hiding nor do forensic tools reveal the hidden data.

4.9. Forging timestamps

Timestamps play a crucial role in keeping track of every possible access, modification, etc. done to the system resources [27]. Forging timestamps can disrupt the forensic investigation from the very first stages. We created a test directory as shown in Figure 13.

We then selected a target file *foo.txt*, and changed its timestamp value using the touch utility as shown in Figure 14. When the file system image was analysed using BFM and FTK, both BFM and FTK were unable to detect any kind of tampering with the timestamp. We further used *hex* utilities to search for older timestamps and the search was unsuccessful. Btrfs supports nanosecond resolution timestamps for increased accuracy, which are central for timeline reconstruction of events in a forensic investigation. This implies that timestamp forgery on Btrfs can go undetected as it doesn't preserve the provenance of MAC-DTS, which is very unlikely for an advanced file system such as Btrfs.

4.10. Modifying magic-numbers

Deleting or changing file system signatures can significantly mislead the forensic investigation, forcing investigators to look for alternative analysis techniques, enough to

```
[root@localhost btrfsTest]# ls -l
total 8
-rw-r--r--. 1 root root 16 Oct  2 02:34 bar.txt
-rw-r--r--. 1 root root 33 Oct  2 02:33 foo.txt
-rw-r--r--. 1 root root  0 Oct  2 02:35 mac.txt
```

Figure 13. Output of ls command before timestamp tampering.

```
[root@localhost btrfsTest]# ls -l
total 12
-rw-r--r--. 1 root root  16 Oct  2 02:34 bar.txt
-rw-r--r--. 1 root root  33 Oct 31  1988 foo.txt
-rw-r--r--. 1 root root   0 Oct  2 02:37 mac.txt
```

Figure 14. Output of ls command after timestamp tampering.

frustrate or slow the entire investigation. In Btrfs, the magic number is contained within every copy of the Superblock. Various tools facilitate the modification of magic numbers on file systems. The wipefs is a Linux-based tool capable of wiping magic numbers on a Btrfs volume. It not only erases file system signatures, but also partition-table magic strings without affecting file system on the device. We used wipefs to remove all magic strings that are visible, as shown in Figure 15.

After successful wiping of magic strings, the partition images were analysed using BFM, FTK and btrfsck. Both BFM and btrfsck reported that the file system is damaged and were unable to verify the file system signature. Although btrfsck is the default checking and rescue utility of Btrfs, all attempts at recovering the magic strings failed. From this we conclude that forensic tools and btrfsck need to be updated to look deeper into file system data structures so to make an educated guess about the file system type.

4.11. Sparse files

Sparse files have different logical file sizes (maximum size) and physical file sizes (space allocated on disk). Btrfs's maximum theoretical limit on the size of individual files is 16 EB but practically it is limited to 7 EB. Figure 16 presents a Linux console procedure in which two 7 EB sparse files are created on a Btrfs file system using the truncate and dd utilities.

```
00010000  fe 29 5d f1 00 00 00 00  00 00 00 00 00 00 00 00  |.)]..............|
00010010  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
00010020  5f 28 54 01 3d 92 44 96  b8 4f 48 09 61 38 f5 48  |_(T.=.D..OH.a8.H|
00010030  00 00 01 00 00 00 00 00  01 00 00 00 00 00 00 00  |................|
00010040  00 00 00 00 00 00 00 00  0f 00 00 00 00 00 00 00  |................|
*
04000040  00 00 00 00 00 00 00 00  0f 00 00 00 00 00 00 00  |................|
```

Figure 15. Removal of Btrfs signature after executing wipefs.

```
//Creating 5MB sparse file using Truncate Utility
[root@localhost btrfsTest]# truncate -s 7E sparseT.txt

//Creating 5MB sparse file using dd Utility
[root@localhost btrfsTest]# dd if=/dev/zero of=./sparseDD.txt bs=1 count=0 seek=7E
0+0 records in
0+0 records out
0 bytes (0 B) copied, 0.000779762 s, 0.0 kB/s
```

Figure 16. Procedure for creating a sparse file on Btrfs.

```
[root@localhost btrfsTest]# ls -lsh
total 4.0K
4.0K  -rw-r--r--. 1 root root  28   Oct 19 01:42  foo.txt
0     -rw-r--r--. 1 root root  7.0E Oct 19 01:59  sparseT.txt
0     -rw-r--r--. 1 root root  7.0E Oct 19 02:05  sparseDD.txt
```

Physical Size

Logical Size

Figure 17. ls command showing physical and logical sizes of a sparse file.

It should also be noted that the size of the partition on which the files are created is limited to 20 MB for better understanding of the power of sparse files. Figure 17 shows the physical and logical sizes of the sparse files created.

Unfortunately, when forensic tools attempt to open sparse files as regular files, the file system transparently initializes and allocates new blocks to the blocks of a sparse file that are mapped to Block 0. In this way, a small sparse file (say 1 MB) can balloon to a large regular file (say 1 GB) during forensic analysis. On one hand, this behaviour may lead to wiping of evidence that may exist in unallocated blocks. On the other hand, a large sparse file may expand beyond the capacity of the disk, forcing the forensic tool to crash and stop working. We encountered both scenarios in our experiments.

Btrfs is one of the few file systems that supports file sizes in Exabyte scale. This can cause significant confusion among forensic investigators while identifying sparse files based on their size. Such actions may prove disastrous if investigators fail to distinguish sparse files from normal ones.

4.12. Mounting on non-empty directories

Mounting a file system on a non-empty directory that contains the forensic evidence is one easy and effective file system anti-forensic technique. It works by storing information in ordinary files and sub-directories, and then mounting an existing (or newly created) file system onto this directory. This causes the data stored inside the mount-point directory to be hidden by the mounted file system.

On a Btrfs file system, btrfsck is able to detect that the device is mounted with some other directory. This implies that hiding data this way can be easily detected using btrfsck.

4.13. Comments on compression and encryption

Btrfs supports transparent file compression at *extent*-level. In addition, compression is done on a file-by-file basis. This implies that we can have a single Btrfs mount point with some files that are uncompressed, some that are compressed with LZO, and some with ZLIB with no means of validating the compression mode of the file. On the other hand, current digital forensic techniques are inadequate to deal with such a significant mix of various compression techniques and states [28].

Btrfs does not yet provide built-in encryption functionality, but works safely with partition encryption techniques such as dm_crypt and supports full disk encryption (FDE) [29] as well. This creates new obstacles for forensic investigators due to their pre-boot authentication prerequisite which makes access to any data impossible unless

the necessary credentials are supplied [29]. In addition, Btrfs can be encrypted using a stacked file system such as *ecryptfs* [30] or a fuse-based file system such as *encfs* as an encrypting layer on top of file system. This implies that an investigator will have to determine the level and the extent of encryption before going on with the decryption process. Forensic investigators have to consider all the possible encryption scenarios, i.e. full-encryption, partial-encryption and no-encryption. Hence, every piece of data needs to be considered both in encrypted and unencrypted form for forensic analysis. This is highly time-consuming and impedes the evidence retrieval process.

5. Conclusion and recommendations

Btrfs is the most advanced file system offered by Linux that is expected to work well on devices as small as smartphones and as large as enterprise servers. Therefore, knowing its anti-forensic capabilities can help in a reliable and effective forensic investigation in the case of a breach or intrusion. This paper presents an analysis of anti-forensic capabilities of Btrfs. The results suggest that the design policy of Btrfs is such that any tampering with critical file system data structures results in file system corruption. This prohibits secure data-deletion of small files and makes it difficult to securely wipe large files, and forbids using reserved locations in Superblock and btrfs_root_item data structures (which are found throughout the file system) and slack space in leaf and internal nodes for data hiding. This implies that it is imperative for a forensic investigator to analyse a corrupt Btrfs volume for such anti-forensic attacks. Unfortunately, Btrfs still shares some weaknesses of its predecessors. These include no support from forensic tools to identify tampering with MAC-DTS and magic strings, and to reveal hidden data in file and volume slack. A forensic investigator must perform manual examination of Btrfs volume for such possible scenarios, and strides must be made to augment forensic tools to handle them automatically. Therefore, the following recommendations can be made in investigation of Btrfs volumes to deal with any anti-forensics attempts:

(1) Wiping data securely on Btrfs is a tedious task and requires technical skill. Furthermore, securely deleting small files of the order of leaf size is highly prone to file system corruption. Therefore, a corrupt Btrfs volume should be analysed for possible remnants of wiping small sized files.
(2) Reserved locations in Superblock and btrfs_root_item data structures of Btrfs can't be used for data hiding. In the former, the data are lost when Superblock is updated (which happens frequently). In the latter, hiding data in reserved locations causes file system corruption. Therefore, forensic tools must analyse a corrupt Btrfs volume for possible hidden data in btrfs_root_item data structures.
(3) Leaf slack and internal-node slack spaces are characteristic features of Btrfs. Hiding data in these spaces renders the file system corrupt. Therefore, a corrupt Btrfs volume must also be analysed for possible hidden data in leaf and internal-node slack.
(4) Boot sector, file slack of *regular-extents*, and volume slack in Btrfs can hide data without corrupting the file system. Unfortunately, these locations are trivially ignored by forensic tools, and hence strides must be made to overcome this loophole.

(5) Unfortunately, for an advanced file system such as Btrfs, the lack of support for provenance of MAC-DTS (due to its in-place updation policy) offers no help to forensic investigators to detect (and fix) timestamp forgery.
(6) Although btrfsck is able to detect mounted file systems, along with forensic tools it fails to recover file system magic strings as well as not being able to handle sparse files gracefully. Therefore, forensic tools must be updated to overcome this pitfall.

Disclosure statement

No potential conflict of interest was reported by the authors.

ORCID

Mohamad Ahtisham Wani http://orcid.org/0000-0002-7635-1718
Wasim Ahmad Bhat http://orcid.org/0000-0002-6043-1609

References

1. Dahbur K, Mohammad B. The anti-forensics challenge. Proceedings of the 2011 International Conference on Intelligent Semantic Web-Services and Applications, Amman, Jordan. ACM; 2011. p. 14.
2. Dezfoli FN, Dehghantanha A, Mahmoud R, Sani NFBM, Daryabar F. Digital forensic trends and future. Int J Cyber-Security Digital Forensics. 2013;2:48–76.
3. Shariati M, Dehghantanha A, Choo K-KR. SugarSync forensic analysis. Aust J Forensic Sci. 2016;48:95–117. doi:10.1080/00450618.2015.1021379
4. Daryabar F, Dehghantanha A, Eterovic-Soric B, Choo K-KR. Forensic investigation of OneDrive, Box, GoogleDrive and Dropbox applications on Android and iOS devices. Aust J Forensic Sci. 2016;48:615–642. doi:10.1080/00450618.2015.1110620
5. Norouzizadeh Dezfouli F, Dehghantanha A, Eterovic-Soric B, Choo K-KR. Investigating social networking applications on smartphones detecting Facebook, Twitter, LinkedIn and Google + artefacts on Android and iOS platforms. Aust J Forensic Sci. 2016;48:469–488. doi:10.1080/00450618.2015.1066854
6. Rodeh O, Bacik J, Mason C. BTRFS: the Linux B-tree filesystem. ACM Trans Storage. 2013;9:9.
7. Newsham T, Palmer C, Stamos A, Burns J. Breaking forensics software: weaknesses in critical evidence collection. Proceedings of the 2007 Black Hat Conference, Las Vegas, USA. 2007.
8. Cifuentes J, Cano J. Analysis and implementation of anti-forensics techniques on ZFS. IEEE Lat Am Trans. 2012;10:1757–1766. doi:10.1109/TLA.2012.6222582
9. Srinivasan A, Wu J. Duplicate file names-a novel steganographic data hiding technique. International Conference on Advances in Computing and Communications, Kochi, India. Springer; 2011. p. 260–268.
10. Srinivasan A, Kolli S, Wu J. Steganographic information hiding that exploits a novel file system vulnerability. Int J Secur Netw. 2013;8:82–93. doi:10.1504/IJSN.2013.055942
11. Huebner E, Bem D, Wee CK. Data hiding in the NTFS file system. Digital Invest. 2006;3:211–226. doi:10.1016/j.diin.2006.10.005
12. Cho G-S. Development of an anti-forensic tool for hiding message in a directory index of NTFS. Internet Security (WorldCIS), 2015 World Congress on. IEEE, Dublin, Ireland; 2015. p. 144–145.
13. Hsu F-H, Wu M-H, Ou S-C, Wang S-J. Data concealments with high privacy in new technology file system. J Supercomput. 2016;72:120–140. doi:10.1007/s11227-015-1492-y

14. Piper S, Davis M, Manes G, Shenoi S. Detecting hidden data in ext2/ext3 file systems. Advances in digital forensics, Orlando, FL, USA. Springer; 2005. p. 245–256.
15. Eckstein K, Jahnke M. Data hiding in journaling file systems. DFRWS, New Orleans, LA, USA. 2005.
16. G¨Obel T, Baier H. Anti-forensics in ext4: on secrecy and usability of timestamp-based data hiding. Digital Investig. 2018;24:S111–S120. doi:10.1016/j.diin.2018.01.014
17. Srinivasan A, Nazaraj ST, Stavrou A. HIDEINSIDE - A novel randomized & encrypted antiforensic information hiding. Computing, Networking and Communications (ICNC), 2013 International Conference on. IEEE, San Diego, CA, USA; 2013. p. 626–631.
18. Savoldi A, Gubian P. SIM and USIM filesystem: a forensics perspective. Proceedings of the 2007 ACM symposium on Applied computing. ACM, Seoul, Korea; 2007. p. 181–187.
19. Jang D-I, Hwang G-JAH, Kim K. Understanding anti-forensic techniques with timestamp manipulation. Information Reuse and Integration (IRI), 2016 IEEE 17th International Conference on. IEEE, Pittsburgh, Pennsylvania, USA; 2016. p. 609–614.
20. Lees C. Determining removal of forensic artefacts using the USN change journal. Digital Investig. 2013;10:300–310. doi:10.1016/j.diin.2013.10.002
21. Rogers M, Lockheed M Anti-forensics *Center for Education and Research in Information Assurance & Security (CERIAS), Department of Information and Computer Technology, Purdue University*. 2005.
22. Conlan K, Baggili I, Breitinger F. Anti-forensics: furthering digital forensic science through a new extended, granular taxonomy. Digital Investig. 2016;18:S66–S75. doi:10.1016/j.diin.2016.04.006
23. Martini B, Choo K-KR. An integrated conceptual digital forensic framework for cloud computing. Digital Investig. 2012;9:71–80. doi:10.1016/j.diin.2012.07.001
24. Bhat WA, Quadri SMK RestFS: secure data deletion using reliable & efficient stackable file system. Applied Machine Intelligence and Informatics (SAMI), 2012 IEEE 10th International Symposium on. IEEE, Herl'any, Slovakia; 2012. p. 457–462.
25. Bhat WA, Quadri SMK. After-deletion data recovery: myths and solutions. Comput Fraud Secur. 2012;2012:17–20. doi:10.1016/S1361-3723(12)70032-5
26. Bhat Wasim A. Achieving efficient purging in transparent per-file secure wiping extensions. Handbook of Research on Security Considerations in Cloud Computing. IGI Global; 2015. p. 345–357.
27. Bhat WA, Quadri SMK. POSTER: dr. Watson provides data for postbreach analysis. Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security. ACM, Berlin, Germany; 2013. p. 1445–1448.
28. Beebe Nicole L, Stacy SD, Stuckey D. Digital forensic implications of ZFS. Digital Invest. 2009;6:S99–S107. doi:10.1016/j.diin.2009.06.006
29. Casey E, Stellatos GJ. The impact of full disk encryption on digital forensics. ACM SIGOPS Operating Syst Rev. 2008;42:93–98. doi:10.1145/1368506.1368519
30. Halcrow MA. eCryptfs: an enterprise-class encrypted filesystem for linux. Proceedings of the 2005 Linux Symposium, Ottawa, Canada. Vol. 1. 2005. p. 201–218.